Pixel Battle — Timeline Script (Sub-project D) — Implementation Plan

Timeline Script (Sub-project D) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Replace condition-driven `ScriptDriver` with absolute-timeline `TimelineDriver`; convert the 5 existing fight scripts to timeline format with prose annotation.

Architecture: New `TimelineDriver` reads two parallel `{t, do}` event lists (one per character) and emits engine actions on the clock. Conflict policy = delay-and-slide: when an actor can't act on time, the rest of THAT character's events shift forward together. Engine untouched except for a small `pending_cast_skill_id` channel that lets `cast:<skill_id>` events name a specific skill (bypassing the engine's `affordable[0]` selection).

Tech Stack: Python 3, PyYAML, pytest, the existing pixel_battle engine.

Spec: `docs/superpowers/specs/2026-05-24-pixel-battle-timeline-script-design.md`. Read it before implementing — it covers the why for every decision in this plan.

File Structure

Create: - `pixel_battle/script/timeline_format.py` — `TimelineEvent` + `Timeline` dataclasses - `pixel_battle/script/timeline_loader.py` — `load_timeline_text` / `load_timeline` (YAML → `Timeline`) -

`pixel_battle/script/timeline_driver.py` — `TimelineDriver.decide(battle) -> (int, int)` – `pixel_battle/scripts/dump_timeline.py` — one-off conversion tool (legacy YAML → timeline trace YAML) – `pixel_battle/data/scripts/legacy/` — directory for archived legacy condition scripts – `pixel_battle/tests/test_timeline_format.py` – `pixel_battle/tests/test_timeline_loader.py` – `pixel_battle/tests/test_timeline_driver.py` – `pixel_battle/tests/test_dump_timeline.py` – `pixel_battle/tests/test_timeline_library.py` – `pixel_battle/tests/test_skill_id_pending.py`

Modify: – `pixel_battle/engine/character.py` — add `pending_cast_skill_id: Optional[str] = None` – `pixel_battle/engine/battle.py` — `_start_attack_with_kind` reads/consumes `pending_cast_skill_id` – `pixel_battle/script/loader.py` — add `load_fight_file(path)` dispatch (timeline vs legacy) – `pixel_battle/rl/play_scripted.py` — use `load_fight_file`, drive with either driver type – `pixel_battle/data/scripts/01_..._05_*.yaml` (×5) — replaced by new timeline-format versions; legacy archived to `legacy/` – `pixel_battle/tests/test_script_library.py` — parametrize over `legacy/` instead of `data/scripts/`

Task 1: TimelineEvent + Timeline dataclasses

Files: – Create: `pixel_battle/script/timeline_format.py` – Test: `pixel_battle/tests/test_timeline_format.py`

- [ ] **Step 1: Write the failing test**

```python
# pixel_battle/tests/test_timeline_format.py
from pixel_battle.script.timeline_format import TimelineEvent,
        Timeline


def test_event_holds_fields():
    ev = TimelineEvent(t=3500, action_int=5,
        raw_do="cast:light_binding",
                       skill_id="light_binding")
    assert ev.t == 3500
    assert ev.action_int == 5
    assert ev.raw_do == "cast:light_binding"
    assert ev.skill_id == "light_binding"


def test_event_skill_id_defaults_to_none():
    ev = TimelineEvent(t=500, action_int=2, raw_do="advance")
```

```
    assert ev.skill_id is None


def test_timeline_holds_two_lists():
    left = [TimelineEvent(t=0, action_int=0, raw_do="idle")]
    right = [TimelineEvent(t=500, action_int=1, raw_do="retreat")]
    tl = Timeline(name="x", left="garen", right="lux",
                  duration_ms=10000, left_events=left,
        right_events=right)
    assert tl.name == "x"
    assert tl.left == "garen"
    assert tl.right == "lux"
    assert tl.duration_ms == 10000
    assert tl.left_events == left
    assert tl.right_events == right
```

- [ ] **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_timeline_format.py -v`
Expected: FAIL with `ModuleNotFoundError: No module named 'pixel_battle.script.timeline_format'`.

- [ ] **Step 3: Write the dataclasses**

```
# pixel_battle/script/timeline_format.py
"""Dataclasses for the absolute-timeline fight format (Sub-project
        D)."""
from __future__ import annotations
from dataclasses import dataclass
from typing import List, Optional


@dataclass
class TimelineEvent:
    """One scheduled action on a character's timeline."""
    t: int                       # ms from match start
    action_int: int              # engine action 0..10
    raw_do: str                  # the source `do` verb (for debug /
        trace)
    skill_id: Optional[str] = None   # set when raw_do is "cast:
        <skill_id>"


@dataclass
class Timeline:
    """A loaded timeline-format fight: two parallel event lists."""
    name: str
    left: str                        # left character id
    right: str                       # right character id
    duration_ms: int                 # author-declared expected fight
        length
```

```
    left_events: List[TimelineEvent]
    right_events: List[TimelineEvent]
```

- [ ] **Step 4: Run test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_timeline_format.py -v`
Expected: PASS — 3/3.

- [ ] **Step 5: Commit**

```
git add pixel_battle/script/timeline_format.py
        pixel_battle/tests/test_timeline_format.py
git commit -m "feat(pixel-battle/script): timeline format dataclasses"
```

Task 2: Engine `pending_cast_skill_id` channel

Lets `cast:<skill_id>` events name a specific skill, bypassing the engine's `affordable[0]` selection. The driver sets `char.pending_cast_skill_id = "<id>"` before returning the action int; the engine consumes the field in `_start_attack_with_kind`.

Files: – Modify: `pixel_battle/engine/character.py` (add field, reset) – Modify: `pixel_battle/engine/battle.py::_start_attack_with_kind` (consume field) – Test: `pixel_battle/tests/test_skill_id_pending.py`

- [ ] **Step 1: Write the failing test**

```
# pixel_battle/tests/test_skill_id_pending.py
"""Verify that pre-setting `pending_cast_skill_id` on a character
        makes
`_start_attack_with_kind` select that specific skill rather than
`affordable[0]` / `first off-cd`."""
from pixel_battle.engine.character import Character
from pixel_battle.engine.skill import SkillType
from pixel_battle.engine.battle import Battle
from pixel_battle.engine.rng import BattleRNG


def _new_battle():
    # Lux has light_binding (cd) and prismatic_barrier (cd, self-
        shield).
    # Engine default = first off-cd CD skill (i.e. light_binding by
        JSON order).
    # We will test: setting pending_cast_skill_id =
        "prismatic_barrier"
    # makes the engine pick prismatic_barrier instead.
    lux = Character.from_id("lux")
    garen = Character.from_id("garen")
    b = Battle(lux, garen, rng=BattleRNG(seed=0))
```

```
    # advance past STARTING and clear last_attack guard
    b.elapsed_ms = 5000
    lux.last_attack_ms = -10000
    return b, lux, garen


def test_pending_cast_overrides_affordable_first():
    b, lux, garen = _new_battle()
    # Default behaviour: pick first off-cd CD skill (light_binding).
    b._start_attack_with_kind(lux, garen, "cooldown")
    assert lux.attack_used_kind is not None
    assert lux.attack_used_kind.id == "light_binding"


def test_pending_cast_picks_named_skill():
    b, lux, garen = _new_battle()
    # Reset to attack-ready
    lux.attack_used_kind = None
    lux.action_state = "idle"
    lux.attack_phase = "none"
    lux.last_attack_ms = -10000
    # Pre-set the named skill the driver wants
    lux.pending_cast_skill_id = "prismatic_barrier"
    b._start_attack_with_kind(lux, garen, "cooldown")
    assert lux.attack_used_kind is not None
    assert lux.attack_used_kind.id == "prismatic_barrier"
    # Field is consumed (cleared) after the attack starts
    assert lux.pending_cast_skill_id is None


def test_pending_cast_unknown_id_noop():
    b, lux, garen = _new_battle()
    lux.attack_used_kind = None
    lux.action_state = "idle"
    lux.attack_phase = "none"
    lux.last_attack_ms = -10000
    lux.pending_cast_skill_id = "no_such_skill_xyz"
    b._start_attack_with_kind(lux, garen, "cooldown")
    # Named skill not in the cd list → no-op (do NOT silently fall
        back to affordable[0])
    assert lux.action_state == "idle"
    assert lux.attack_used_kind is None
    # Field is consumed even on no-op so a stale id can't fire next
        tick
    assert lux.pending_cast_skill_id is None
```

- [ ] **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_skill_id_pending.py -v`
Expected: FAIL — `AttributeError: 'Character' object has no attribute 'pending_cast_skill_id'`.

- [ ] **Step 3: Add the field to `Character`**

Open `pixel_battle/engine/character.py`. Find the `Character` dataclass and add the field next to `last_attack_ms` (or wherever the per-tick mutable fields live):

```
# inside @dataclass Character:
pending_cast_skill_id: Optional[str] = None
```

In `reset_physics` (or the equivalent per-match reset method), add:

```
self.pending_cast_skill_id = None
```

- [ ] **Step 4: Make `_start_attack_with_kind` consume the field**

In `pixel_battle/engine/battle.py`, replace the existing `_start_attack_with_kind` body skill-selection block. Read the current method first; then for each kind branch (`basic`, `cooldown`, `special`, `kick`), apply the named-skill override:

```
def _start_attack_with_kind(self, char, opp, kind: str) -> None:
    """RL-friendly attack initiator.  Respects
        `char.pending_cast_skill_id`:
    if set, the engine picks that specific skill within the SkillType
        list
    (or no-ops if the named skill is not on the character or not
        available).
    The field is consumed (cleared) on every call, success or no-op,
        so a
    stale id cannot fire on a later tick."""
    if char.action_state in ("attacking", "hit_stagger", "ko"):
        char.pending_cast_skill_id = None
        return
    if self.elapsed_ms < char.last_attack_ms +
        char.attack_interval_ms:
        char.pending_cast_skill_id = None
        return

    explicit_id = char.pending_cast_skill_id
    char.pending_cast_skill_id = None     # consumed regardless of
        outcome

    skill = None
    if kind == "basic":
        basics = char.skills_of_type(SkillType.BASIC)
        skill = (next((s for s in basics if s.id == explicit_id),
        None)
                 if explicit_id else basics[0])
        char.attack_anim_hint = "jab"
    elif kind == "cooldown":
        cd_skills = char.skills_of_type(SkillType.COOLDOWN)
        available = [s for s in cd_skills
```

```
                    if char.skill_off_cooldown(s, self.elapsed_ms)]
        if explicit_id:
            skill = next((s for s in available if s.id ==
        explicit_id), None)
        elif available:
            skill = available[0]
        char.attack_anim_hint = "cooldown"
    elif kind == "special":
        specials = char.skills_of_type(SkillType.SPECIAL)
        affordable = [s for s in specials if char.mp >= s.mp_cost]
        if explicit_id:
            skill = next((s for s in affordable if s.id ==
        explicit_id), None)
        elif affordable:
            skill = affordable[0]
        char.attack_anim_hint = "special"
    elif kind == "kick":
        basics = char.skills_of_type(SkillType.BASIC)
        skill = (next((s for s in basics if s.id == explicit_id),
        None)
                 if explicit_id else basics[0])
        char.attack_anim_hint = "kick"
    else:
        return

    if skill is None:
        return    # named skill not available or unknown kind → no-op

    # Existing body — start the attack with the selected skill
    char.attack_used_kind = skill
    char.attack_phase = "windup"
    char.attack_phase_t = 0
    char.action_state = "attacking"
    char.vel_x = 0.0

    if skill.applies is not None and skill.applies.target == "self":
        self._apply_effect(char, skill.applies)

    if skill.skill_type in (SkillType.COOLDOWN, SkillType.SPECIAL):
        self._emit(
            EventType.ATTACK_WINDUP,
            actor=char.id,
            extra={"skill_id": skill.id,
                   "skill_type": skill.skill_type.value,
                   "vfx": skill.vfx},
        )
        self._apply_cast_movement(char, opp, skill)
```

- [ ] **Step 5: Run new test + regression suite**

Run: `python -m pytest pixel_battle/tests/test_skill_id_pending.py -v`
Expected: PASS — 3/3.

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 381 + 3 = 384 passed; the engine change is backward-compatible (default `None` → same behaviour as before).

- [ ] **Step 6: Commit**

```
git add pixel_battle/engine/character.py pixel_battle/engine/battle.py
        pixel_battle/tests/test_skill_id_pending.py
git commit -m "feat(pixel-battle/engine): pending_cast_skill_id
        channel for named skill selection"
```

Task 3: TimelineLoader

YAML → `Timeline`. Validates verbs, skill IDs, monotonic `t`, required fields.

Files: – Create: `pixel_battle/script/timeline_loader.py` – Test: `pixel_battle/tests/test_timeline_loader.py`

- [ ] **Step 1: Write the failing test**

```python
# pixel_battle/tests/test_timeline_loader.py
import pytest
from pixel_battle.script.timeline_loader import (
    load_timeline_text, TimelineLoadError,
)


_GOOD = """
name: "test fight"
left: garen
right: lux
duration_ms: 10000
left_timeline:
  - {t: 0, do: idle}
  - {t: 500, do: advance}
  - {t: 3000, do: "attack:basic"}
right_timeline:
  - {t: 0, do: idle}
  - {t: 800, do: retreat}
  - {t: 3000, do: "cast:light_binding"}
"""


def test_parses_valid_yaml():
    tl = load_timeline_text(_GOOD)
    assert tl.name == "test fight"
    assert tl.left == "garen"
    assert tl.right == "lux"
    assert tl.duration_ms == 10000
```

```
    assert len(tl.left_events) == 3
    assert len(tl.right_events) == 3
    # `do: advance` → action_int 2, no skill_id
    assert tl.left_events[1].action_int == 2
    assert tl.left_events[1].skill_id is None
    # `do: cast:light_binding` → action_int 5 (cooldown), skill_id set
    assert tl.right_events[2].action_int == 5
    assert tl.right_events[2].skill_id == "light_binding"


def test_unknown_do_verb_rejected():
    bad = _GOOD.replace("do: advance", "do: wiggle")
    with pytest.raises(TimelineLoadError, match="unknown do verb"):
        load_timeline_text(bad)


def test_unknown_skill_id_rejected():
    bad = _GOOD.replace("cast:light_binding", "cast:no_such_skill")
    with pytest.raises(TimelineLoadError, match="unknown skill id"):
        load_timeline_text(bad)


def test_skill_not_on_character_rejected():
    # Garen does not have light_binding (it's a Lux skill). If we put
        a Lux-only
    # skill on Garen's timeline, the loader must reject it.
    bad = _GOOD.replace("right_timeline", "GARENTL")  # gut the right
        block
    # Insert a cast for the LEFT (Garen) using a Lux-only skill
    bad = bad.replace("do: \"attack:basic\"", "do:
        \"cast:light_binding\"")
    with pytest.raises(TimelineLoadError, match="not a skill of"):
        load_timeline_text(bad)


def test_nonmonotonic_t_rejected():
    bad = _GOOD.replace("t: 3000, do: \"attack:basic\"",
                        "t: 100, do: \"attack:basic\"")
    with pytest.raises(TimelineLoadError, match="non-monotonic"):
        load_timeline_text(bad)


def test_missing_required_key():
    with pytest.raises(TimelineLoadError, match="missing required
        key"):
        load_timeline_text("name: x\nleft: garen\nright: lux\n")


def test_unknown_character_rejected():
    bad = _GOOD.replace("left: garen", "left: nobody_here")
    with pytest.raises(TimelineLoadError, match="unknown character"):
        load_timeline_text(bad)
```

```
def test_negative_t_rejected():
    bad = _GOOD.replace("t: 500, do: advance",
                        "t: -100, do: advance")
    with pytest.raises(TimelineLoadError, match="negative"):
        load_timeline_text(bad)
```

- [ ] **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_timeline_loader.py -v`
Expected: FAIL with `ModuleNotFoundError: No module named 'pixel_battle.script.timeline_loader'`.

- [ ] **Step 3: Write the loader**

```
# pixel_battle/script/timeline_loader.py
"""Parse + validate a timeline-format fight YAML into a `Timeline`."""
from __future__ import annotations
import json
from pathlib import Path
from typing import List

import yaml

from pixel_battle.engine.character import DATA_PATH
from pixel_battle.engine.skill import SkillType
from pixel_battle.script.loader import DO_VERBS    # 11 do-verbs → action ints
from pixel_battle.script.timeline_format import Timeline, TimelineEvent


# `cast:<skill_id>` routes to the same action int as the skill's `attack:<kind>`.
_SKILL_TYPE_TO_ACTION = {
    SkillType.BASIC: 4,
    SkillType.COOLDOWN: 5,
    SkillType.ULTIMATE: 6,
    SkillType.SPECIAL: 7,
}


class TimelineLoadError(ValueError):
    """Raised when a timeline-format fight file is malformed."""


def _load_character_db():
    with open(DATA_PATH, encoding="utf-8") as f:
        return json.load(f)
```

```
def _resolve_do(do: str, char_id: str, db: dict, side: str) -> (int,
        str):
    """Resolve a `do` field to (action_int, skill_id_or_None).

    Raises TimelineLoadError on unknown verb or unknown skill."""
    if do in DO_VERBS:
        return DO_VERBS[do], None
    if not do.startswith("cast:"):
        raise TimelineLoadError(f"{side}: unknown do verb {do!r}")
    skill_id = do.split(":", 1)[1]
    char_data = db[char_id]
    skill_entries = char_data.get("skills", [])
    match = next((s for s in skill_entries if s["id"] == skill_id),
        None)
    if match is None:
        # Disambiguate: unknown across the whole game vs known but not
        on this char
        is_known = any(s["id"] == skill_id
                       for c in db.values() for s in c.get("skills",
        []))
        if is_known:
            raise TimelineLoadError(
                f"{side}: {skill_id!r} is not a skill of {char_id!r}")
        raise TimelineLoadError(f"{side}: unknown skill id
        {skill_id!r}")
    stype = SkillType(match["type"])
    action_int = _SKILL_TYPE_TO_ACTION.get(stype)
    if action_int is None:
        raise TimelineLoadError(
            f"{side}: skill {skill_id!r} has unroutable type
        {stype.value!r}")
    return action_int, skill_id


def _parse_timeline(raw, char_id: str, db: dict, side: str) ->
        List[TimelineEvent]:
    if not isinstance(raw, list):
        raise TimelineLoadError(f"{side}_timeline must be a list")
    events: List[TimelineEvent] = []
    last_t = -1
    for i, item in enumerate(raw):
        if not isinstance(item, dict) or "t" not in item or "do" not
        in item:
            raise TimelineLoadError(
                f"{side}_timeline[{i}] needs 't' and 'do'")
        t = item["t"]
        if not isinstance(t, int):
            raise TimelineLoadError(
                f"{side}_timeline[{i}]: t must be an integer (ms), got
        {t!r}")
        if t < 0:
            raise TimelineLoadError(
                f"{side}_timeline[{i}]: t is negative ({t})")
```

```
        if t < last_t:
            raise TimelineLoadError(
                f"{side}_timeline[{i}]: non-monotonic t "
                f"({t} < previous {last_t})")
        do = str(item["do"])
        action_int, skill_id = _resolve_do(
            do, char_id, db, f"{side}_timeline[{i}]")
        events.append(TimelineEvent(
            t=t, action_int=action_int, raw_do=do, skill_id=skill_id))
        last_t = t
    return events


def load_timeline_text(text: str) -> Timeline:
    try:
        data = yaml.safe_load(text)
    except yaml.YAMLError as e:
        raise TimelineLoadError(f"invalid YAML: {e}") from e
    if not isinstance(data, dict):
        raise TimelineLoadError("timeline file must be a YAML
        mapping")
    for key in ("name", "left", "right", "duration_ms",
                "left_timeline", "right_timeline"):
        if key not in data:
            raise TimelineLoadError(f"missing required key: {key!r}")
    db = _load_character_db()
    for side in ("left", "right"):
        if data[side] not in db:
            raise TimelineLoadError(f"unknown character id:
        {data[side]!r}")
    return Timeline(
        name=str(data["name"]),
        left=str(data["left"]), right=str(data["right"]),
        duration_ms=int(data["duration_ms"]),
        left_events=_parse_timeline(
            data["left_timeline"], str(data["left"]), db, "left"),
        right_events=_parse_timeline(
            data["right_timeline"], str(data["right"]), db, "right"),
    )


def load_timeline(path) -> Timeline:
    """Load + validate a timeline-format fight from a YAML file
        path."""
    return load_timeline_text(Path(path).read_text(encoding="utf-8"))
```

- [ ] **Step 4: Run test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_timeline_loader.py -v`
Expected: PASS — 8/8.

- [ ] **Step 5: Commit**

```
git add pixel_battle/script/timeline_loader.py
        pixel_battle/tests/test_timeline_loader.py
git commit -m "feat(pixel-battle/script): timeline loader — parse +
        validate timeline YAML"
```

Task 4: TimelineDriver

The conflict-policy heart of D. Per-character cursors + per-character delay offsets; events fire when `elapsed_ms >= event.t + delay`; if the actor can't act, delay accumulates by one engine tick and event retries next tick. The two character timelines slide independently.

Files: – Create: `pixel_battle/script/timeline_driver.py` – Test: `pixel_battle/tests/test_timeline_driver.py`

- [ ] **Step 1: Write the failing tests**

```python
# pixel_battle/tests/test_timeline_driver.py
from pixel_battle.script.timeline_format import Timeline,
        TimelineEvent
from pixel_battle.script.timeline_driver import TimelineDriver
from pixel_battle.engine.character import Character
from pixel_battle.engine.effects import StatusEffect, ROOT
from pixel_battle.engine.battle import Battle
from pixel_battle.engine.rng import BattleRNG


def _two_char_battle(left_events, right_events, duration_ms=10000):
    left = Character.from_id("garen")
    right = Character.from_id("lux")
    b = Battle(left, right, rng=BattleRNG(seed=0))
    tl = Timeline(name="t", left="garen", right="lux",
                  duration_ms=duration_ms,
                  left_events=left_events, right_events=right_events)
    d = TimelineDriver(tl)
    return b, d


def test_event_fires_at_scheduled_t():
    left = [TimelineEvent(t=500, action_int=2, raw_do="advance")]
    right = [TimelineEvent(t=0, action_int=0, raw_do="idle")]
    b, d = _two_char_battle(left, right)
    # Before scheduled time → both idle
    b.elapsed_ms = 0
    assert d.decide(b) == (0, 0)
    b.elapsed_ms = 499
    assert d.decide(b) == (0, 0)
    # At/after scheduled time AND actor can act → fires
    b.elapsed_ms = 500
```

```
    left_act, right_act = d.decide(b)
    assert left_act == 2     # advance
    assert right_act == 0    # idle (script exhausted on right)


def test_cursor_advances_only_once_per_event():
    left = [TimelineEvent(t=0, action_int=2, raw_do="advance"),
            TimelineEvent(t=1000, action_int=1, raw_do="retreat")]
    right = []
    b, d = _two_char_battle(left, right)
    b.elapsed_ms = 0
    assert d.decide(b)[0] == 2     # advance fires
    b.elapsed_ms = 16
    assert d.decide(b)[0] == 0     # cursor moved on; next event not
        due yet
    b.elapsed_ms = 1000
    assert d.decide(b)[0] == 1     # retreat fires


def test_event_delays_when_actor_in_attack_phase():
    # Left has an event at t=0; we mark left as mid-attack so it can't
        act.
    left = [TimelineEvent(t=0, action_int=2, raw_do="advance"),
            TimelineEvent(t=100, action_int=1, raw_do="retreat")]
    right = []
    b, d = _two_char_battle(left, right)
    b.left.action_state = "attacking"
    b.left.attack_phase = "windup"
    b.elapsed_ms = 0
    assert d.decide(b)[0] == 0     # can't act → idle emitted, delay
        accumulates
    b.elapsed_ms = 16
    assert d.decide(b)[0] == 0
    # Free the actor; the FIRST event should now fire, not be skipped
    b.left.action_state = "idle"
    b.left.attack_phase = "none"
    b.elapsed_ms = 32
    assert d.decide(b)[0] == 2     # advance finally fires
    # The second event was scheduled at t=100; with delay 32ms already
    # accumulated, it fires at t>=132 (NOT at t>=100).
    b.elapsed_ms = 100
    assert d.decide(b)[0] == 0     # not yet
    b.elapsed_ms = 132
    assert d.decide(b)[0] == 1     # retreat fires (shifted)


def test_root_blocks_movement_but_not_cast():
    # Movement (advance) is blocked under root; cast is NOT blocked.
    left = [TimelineEvent(t=0, action_int=2, raw_do="advance")]
    right = [TimelineEvent(t=0, action_int=5,
        raw_do="cast:light_binding",
                           skill_id="light_binding")]
```

```
    b, d = _two_char_battle(left, right)
    b.left.effects.append(StatusEffect(kind=ROOT, remaining_ms=2000,
        magnitude=1.0))
    # right is not rooted
    b.elapsed_ms = 0
    left_act, right_act = d.decide(b)
    assert left_act == 0     # movement blocked
    assert right_act == 5    # cast still fires


def test_cast_sets_pending_skill_id_on_character():
    left = [TimelineEvent(t=0, action_int=5,
        raw_do="cast:light_binding",
                          skill_id="light_binding")]
    right = []
    b, d = _two_char_battle([], left)   # put on right (lux) since
        light_binding is Lux's
    # (above: swap order so right has the cast event)
    b.elapsed_ms = 0
    d.decide(b)
    assert b.right.pending_cast_skill_id == "light_binding"


def test_per_character_independence():
    # Left delayed (mid-attack) but right's events fire on schedule.
    left = [TimelineEvent(t=0, action_int=2, raw_do="advance"),
            TimelineEvent(t=500, action_int=2, raw_do="advance")]
    right = [TimelineEvent(t=500, action_int=1, raw_do="retreat")]
    b, d = _two_char_battle(left, right)
    b.left.action_state = "attacking"
    b.left.attack_phase = "windup"
    b.elapsed_ms = 500
    left_act, right_act = d.decide(b)
    assert left_act == 0     # delayed
    assert right_act == 1    # fires on time — right's clock
        unaffected


def test_exhausted_cursor_emits_idle():
    left = [TimelineEvent(t=0, action_int=2, raw_do="advance")]
    right = []
    b, d = _two_char_battle(left, right)
    b.elapsed_ms = 0
    d.decide(b)
    b.elapsed_ms = 5000
    assert d.decide(b) == (0, 0)     # both exhausted → idle
```

- [ ] **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_timeline_driver.py -v`
Expected: FAIL with `ModuleNotFoundError: No module named 'pixel_battle.script.timeline_driver'`.

- [ ] **Step 3: Write the driver**

```
# pixel_battle/script/timeline_driver.py
"""TimelineDriver — plays a Timeline against a Battle, tick by tick.

Per-character cursors + per-character accumulated delay offsets. When
        a
character cannot act at the scheduled time, the entire remaining
        timeline
on THAT character shifts forward together; the other character is
unaffected. See spec §7."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import List

from pixel_battle.script.timeline_format import Timeline,
        TimelineEvent


# Engine tick granularity — match Battle.tick_ms's typical step (the
        renderer
# steps at 1 frame ≈ 16 ms). When the driver can't fire an event, it
        pushes
# the per-character delay by this much so the event retries next tick.
ENGINE_TICK_MS = 16

# Action int constants — kept in sync with DO_VERBS / engine
        `_apply_action`.
_IDLE = 0
_RETREAT = 1
_ADVANCE = 2
_JUMP = 3
_ATK_BASIC = 4
_ATK_CD = 5
_ATK_ULT = 6
_ATK_SPECIAL = 7
_ATK_KICK = 8
_FLASH_IN = 9
_FLASH_BACK = 10

_ATTACK_ACTIONS = frozenset({_ATK_BASIC, _ATK_CD, _ATK_ULT,
        _ATK_SPECIAL, _ATK_KICK})
_MOVE_ACTIONS = frozenset({_RETREAT, _ADVANCE, _JUMP})


@dataclass
class _SideCursor:
    events: List[TimelineEvent]
    index: int = 0
    delay_ms: int = 0

    def peek(self) -> TimelineEvent:
```

```
        return self.events[self.index] if self.index <
        len(self.events) else None

    def advance(self) -> None:
        self.index += 1


def _can_act(char, action_int: int) -> bool:
    """Predicate: would emitting this action right now be wasted?

    Kept deliberately small. Engine's `_apply_action` remains
        authoritative
    on cooldown / mp / range / facing; the driver only checks two
        cases
    where the engine would silently drop the event and the script's
        intent
    is clearly "wait for the block to clear":
      - mid-attack-phase (windup/active/recover): a new
        attack/movement is
        a no-op until the current animation ends.
      - rooted + movement verb: root pins the character; the script
        wants
        movement to fire after root drops, not while it's active.
    Casting and Flash are NOT blocked by root (Flash bypasses root; a
        cast
    issued under root should start windup the moment root expires)."""
    if action_int == _IDLE:
        return True
    if char.action_state in ("attacking", "hit_stagger"):
        return False
    # `pos_y > GROUND_Y - tolerance` -- jumping mid-air. Ground verbs
        (move/
    # attack) are engine no-ops in mid-air; let them wait for landing.
    if char.action_state == "jumping" and action_int in (_MOVE_ACTIONS
        | _ATTACK_ACTIONS):
        return False
    # Root: blocks movement; Flash/cast still allowed
    from pixel_battle.engine.effects import ROOT
    if char.has_effect(ROOT) and action_int in _MOVE_ACTIONS:
        return False
    return True


class TimelineDriver:
    """Drives a Battle from a Timeline. Same call interface as
        ScriptDriver:
    `decide(battle) -> (left_action, right_action)` per tick."""

    def __init__(self, timeline: Timeline):
        self.timeline = timeline
        self._left = _SideCursor(events=timeline.left_events)
        self._right = _SideCursor(events=timeline.right_events)
```

```
    def decide(self, battle):
        left_act = self._decide_side(self._left, battle.left,
        battle.elapsed_ms)
        right_act = self._decide_side(self._right, battle.right,
        battle.elapsed_ms)
        return left_act, right_act

    def _decide_side(self, cursor: _SideCursor, char, elapsed_ms: int)
        -> int:
        ev = cursor.peek()
        if ev is None:
            return _IDLE
        scheduled_t = ev.t + cursor.delay_ms
        if elapsed_ms < scheduled_t:
            return _IDLE
        if not _can_act(char, ev.action_int):
            cursor.delay_ms += ENGINE_TICK_MS
            return _IDLE
        # Fire — set the named-skill channel for cast: events, then
        advance cursor.
        if ev.skill_id is not None:
            char.pending_cast_skill_id = ev.skill_id
        cursor.advance()
        return ev.action_int
```

- [ ] **Step 4: Run test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_timeline_driver.py -v`
Expected: PASS — 7/7.

- [ ] **Step 5: Run full suite**

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 384 + 7 = 391 passed.

- [ ] **Step 6: Commit**

```
git add pixel_battle/script/timeline_driver.py
        pixel_battle/tests/test_timeline_driver.py
git commit -m "feat(pixel-battle/script): TimelineDriver — absolute-
        time action source"
```

Task 5: Loader dispatch + `play_scripted.py` glue

Top-level entry point picks the right driver based on YAML format. `play_scripted.py` now drives either kind.

Files: – Modify: `pixel_battle/script/loader.py` (add `load_fight_file`) – Modify: `pixel_battle/rl/play_scripted.py` (use `load_fight_file`) – Test: `pixel_battle/tests/test_loader_dispatch.py` (new)

- [ ] **Step 1: Write the failing test**

```python
# pixel_battle/tests/test_loader_dispatch.py
import pytest
from pixel_battle.script.loader import load_fight_file, ScriptError
from pixel_battle.script.driver import ScriptDriver
from pixel_battle.script.timeline_driver import TimelineDriver


_TIMELINE_YAML = """
name: tl
left: garen
right: lux
duration_ms: 5000
left_timeline:
  - {t: 0, do: idle}
right_timeline:
  - {t: 0, do: idle}
"""

_LEGACY_YAML = """
name: legacy
left: garen
right: lux
left_script:
  - {do: idle, until: "time>=5000"}
right_script:
  - {do: idle, until: "time>=5000"}
"""

_AMBIGUOUS_YAML = """
name: bad
left: garen
right: lux
left_timeline:
  - {t: 0, do: idle}
right_script:
  - {do: idle, until: "time>=5000"}
"""


def test_dispatches_timeline(tmp_path):
    p = tmp_path / "x.yaml"
    p.write_text(_TIMELINE_YAML)
    driver = load_fight_file(p)
    assert isinstance(driver, TimelineDriver)
```

```
def test_dispatches_legacy(tmp_path):
    p = tmp_path / "x.yaml"
    p.write_text(_LEGACY_YAML)
    driver = load_fight_file(p)
    assert isinstance(driver, ScriptDriver)


def test_ambiguous_rejected(tmp_path):
    p = tmp_path / "x.yaml"
    p.write_text(_AMBIGUOUS_YAML)
    with pytest.raises(ScriptError, match="ambiguous"):
        load_fight_file(p)


def test_unknown_format_rejected(tmp_path):
    p = tmp_path / "x.yaml"
    p.write_text("name: x\nleft: garen\nright: lux\n")
    with pytest.raises(ScriptError, match="neither"):
        load_fight_file(p)
```

- [ ] **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_loader_dispatch.py -v`
Expected: FAIL with `ImportError: cannot import name 'load_fight_file' from 'pixel_battle.script.loader'`.

- [ ] **Step 3: Add `load_fight_file` to `pixel_battle/script/loader.py`**

Append to `pixel_battle/script/loader.py`:

```
def load_fight_file(path):
    """Top-level loader: returns either a ScriptDriver (legacy
        condition
    format) or a TimelineDriver (new timeline format) based on which
    timeline/script keys the YAML contains.

    Detection: presence of `left_timeline` / `right_timeline` keys →
        timeline;
    presence of `left_script` / `right_script` → legacy; mixed =
        ambiguous;
    neither = unknown format."""
    from pixel_battle.script.timeline_loader import load_timeline
    from pixel_battle.script.timeline_driver import TimelineDriver
    from pixel_battle.script.driver import ScriptDriver

    text = Path(path).read_text(encoding="utf-8")
    try:
        data = yaml.safe_load(text)
    except yaml.YAMLError as e:
        raise ScriptError(f"invalid YAML: {e}") from e
    if not isinstance(data, dict):
```

```
        raise ScriptError("fight file must be a YAML mapping")
    has_timeline = ("left_timeline" in data) or ("right_timeline" in
        data)
    has_script = ("left_script" in data) or ("right_script" in data)
    if has_timeline and has_script:
        raise ScriptError(
            f"{path}: ambiguous format — has both timeline and script
        keys")
    if has_timeline:
        return TimelineDriver(load_timeline(path))
    if has_script:
        return ScriptDriver(load_script_text(text))
    raise ScriptError(
        f"{path}: neither timeline (left_timeline/right_timeline) "
        "nor legacy (left_script/right_script) keys present")
```

- [ ] Step 4: Update **`pixel_battle/rl/play_scripted.py`**

Read the current `render_script`. Replace the loader+driver construction:

```
# pixel_battle/rl/play_scripted.py
"""Render a fight from an authored YAML script (no RL policy).

Auto-detects legacy condition scripts and new timeline scripts; the
        loader
returns whichever driver fits the file."""
from __future__ import annotations
import argparse
import os
from pathlib import Path

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame  # noqa: E402

from pixel_battle.rl.env import PixelBattleEnv  # noqa: E402
from pixel_battle.rl.play import _render_fight, WIDTH, HEIGHT, FPS,
        ROOT  # noqa: E402
from pixel_battle.video.recorder import FrameRecorder  # noqa: E402
from pixel_battle.script.loader import load_fight_file  # noqa: E402

OUT_DIR = ROOT / "pixel_battle" / "output" / "scripted"


def _driver_action_source(driver):
    """Adapt any driver with `.decide(battle)` to the
    (env, obs) -> (left_act, right_act) interface used by
        `_render_fight`."""
    def _source(env, _obs):
        return driver.decide(env.battle)
    return _source
```

```python
def render_script(script_path: Path, out_dir: Path = OUT_DIR) -> Path:
    pygame.init()
    pygame.display.set_mode((1, 1))
    out_dir.mkdir(parents=True, exist_ok=True)

    driver = load_fight_file(script_path)
    # Both driver types carry left/right character ids in their loaded
        data
    if hasattr(driver, "script"):
        left_id, right_id = driver.script.left, driver.script.right
    else:
        left_id, right_id = driver.timeline.left,
        driver.timeline.right
    env = PixelBattleEnv(left_id=left_id, right_id=right_id)

    raw = out_dir / f"{script_path.stem}_raw.mp4"
    recorder = FrameRecorder(str(raw), fps=FPS, width=WIDTH,
        height=HEIGHT)
    recorder.start()
    try:
        _render_fight(recorder, _driver_action_source(driver), env,
                      max_seconds=60.0, end_hold_frames=120)
    finally:
        recorder.stop()
    print(f"Scripted render: {raw}")
    return raw


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("script", type=Path, help="path to a fight-script
        YAML")
    args = p.parse_args()
    render_script(args.script)
```

- [ ] **Step 5: Run dispatch test + regression**

Run: `python -m pytest pixel_battle/tests/test_loader_dispatch.py -v` Expected: PASS — 4/4.

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 391 + 4 = 395 passed.

- [ ] **Step 6: Commit**

```
git add pixel_battle/script/loader.py pixel_battle/rl/play_scripted.py
        pixel_battle/tests/test_loader_dispatch.py
git commit -m "feat(pixel-battle/script): top-level load_fight_file
        dispatch — timeline vs legacy"
```

Task 6: `dump_timeline.py` trace tool

Runs a legacy script through the existing `ScriptDriver`+engine, logs every non-idle action emit with its `elapsed_ms`, outputs a candidate timeline YAML.

Files: – Create: `pixel_battle/scripts/dump_timeline.py` – Create: `pixel_battle/scripts/__init__.py` (empty, for package import in test) – Test: `pixel_battle/tests/test_dump_timeline.py`

☐ **Step 1: Write the failing test**

```
# pixel_battle/tests/test_dump_timeline.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pathlib import Path
import yaml

from pixel_battle.scripts.dump_timeline import dump_timeline_for


def test_dumps_a_parseable_timeline(tmp_path):
    legacy =
        Path("pixel_battle/data/scripts/legacy/01_lux_kite_garen.yaml")
    # Skip if the legacy file isn't there yet (Task 7 archives it
        later);
    # for this Task 6 test we point at any existing condition script.
    if not legacy.exists():
        legacy =
        Path("pixel_battle/data/scripts/01_lux_kite_garen.yaml")
    out = tmp_path / "trace.yaml"
    dump_timeline_for(legacy, out)
    assert out.exists()
    data = yaml.safe_load(out.read_text())
    assert "left_timeline" in data
    assert "right_timeline" in data
    # At least the LEFT or RIGHT timeline should have non-idle events
        emitted
    # during the legacy simulation
    total_events = len(data["left_timeline"]) +
        len(data["right_timeline"])
    assert total_events >= 4, (
        f"trace has too few events ({total_events}); "
        "the legacy run should emit at least basic+advance per side")
    # Monotonic t on each side
    for side in ("left_timeline", "right_timeline"):
        ts = [ev["t"] for ev in data[side]]
        assert ts == sorted(ts), f"{side} timestamps not monotonic:
        {ts}"
```

☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_dump_timeline.py -v`
Expected: FAIL with `ModuleNotFoundError: No module named 'pixel_battle.scripts'`.

- [ ] **Step 3: Create `pixel_battle/scripts/__init__.py`**

Empty file:

```
# pixel_battle/scripts/__init__.py
```

- [ ] **Step 4: Write `dump_timeline.py`**

```
# pixel_battle/scripts/dump_timeline.py
"""Convert a legacy condition-script YAML into a candidate timeline
        YAML by
simulating the fight and logging every non-idle action emit per side.

Usage:
    python -m pixel_battle.scripts.dump_timeline \\
        pixel_battle/data/scripts/legacy/01_lux_kite_garen.yaml \\
        pixel_battle/data/scripts/01_lux_kite_garen.yaml

The output is a STARTING POINT — the author rounds timestamps to 100
        ms,
trims misses, and adds prose chapter comments.
"""
from __future__ import annotations
import argparse
import os
from pathlib import Path

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

import yaml

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.script.driver import ScriptDriver
from pixel_battle.script.loader import load_script, DO_VERBS


_ACTION_TO_VERB = {v: k for k, v in DO_VERBS.items()}
TICK_MS = 16
MAX_MS = 60_000


def _engine_action_to_do(action_int: int) -> str:
    """Map action int back to a `do` verb string. Same as DO_VERBS
        reverse."""
    if action_int in _ACTION_TO_VERB:
        return _ACTION_TO_VERB[action_int]
    return f"unknown:{action_int}"
```

```
def dump_timeline_for(legacy_path: Path, out_path: Path) -> None:
    script = load_script(legacy_path)
    driver = ScriptDriver(script)
    env = PixelBattleEnv(left_id=script.left, right_id=script.right)
    env.reset()

    left_events = []
    right_events = []
    last_left_action = 0
    last_right_action = 0
    while env.battle.elapsed_ms < MAX_MS and not env.battle.state.name
        == "KO":
        left_act, right_act = driver.decide(env.battle)
        t = env.battle.elapsed_ms
        # Log a transition into a non-idle action (or any change away
        from idle)
        if left_act != 0 and left_act != last_left_action:
            left_events.append({"t": t, "do":
        _engine_action_to_do(left_act)})
        if right_act != 0 and right_act != last_right_action:
            right_events.append({"t": t, "do":
        _engine_action_to_do(right_act)})
        last_left_action = left_act
        last_right_action = right_act
        env.step(left_act, right_act)

    data = {
        "name": script.name,
        "left": script.left,
        "right": script.right,
        "duration_ms": env.battle.elapsed_ms,
        "left_timeline": left_events,
        "right_timeline": right_events,
    }
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text(
        "# Generated by dump_timeline.py — round timestamps to 100 ms,
        "
        "trim out_of_range emissions, add prose chapter headers.\n"
        + yaml.safe_dump(data, sort_keys=False, allow_unicode=True),
        encoding="utf-8",
    )


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("legacy", type=Path, help="legacy condition-script
        YAML")
    p.add_argument("out", type=Path, help="output timeline YAML")
    args = p.parse_args()
```

```
    dump_timeline_for(args.legacy, args.out)
    print(f"wrote {args.out}")
```

- [ ] **Step 5: Run test to verify it passes**

Run: `python -m pytest pixel_battle/tests/test_dump_timeline.py -v`
Expected: PASS — 1/1.

- [ ] **Step 6: Commit**

```
git add pixel_battle/scripts/__init__.py
        pixel_battle/scripts/dump_timeline.py
        pixel_battle/tests/test_dump_timeline.py
git commit -m "feat(pixel-battle/scripts): dump_timeline — legacy YAML
        to trace candidate"
```

Task 7: Convert the 5 legacy scripts

For each of the 5 scripts: 1. Move the legacy YAML to `pixel_battle/data/scripts/legacy/<name>.yaml`. 2. Run `dump_timeline.py` against the legacy file → produces a raw trace candidate at `pixel_battle/data/scripts/<name>.yaml`. 3. Hand-polish the candidate: round each `t` to the nearest 100 ms; trim consecutive duplicate emissions; add a prose chapter-header block as YAML comments at the top of the file.

Files: – Move: `pixel_battle/data/scripts/01_lux_kite_garen.yaml` → `legacy/01_lux_kite_garen.yaml` (and 02—05 similarly) – Replace: `pixel_battle/data/scripts/01_lux_kite_garen.yaml` (and 02—05) with timeline-format content – Modify: `pixel_battle/tests/test_script_library.py` — point parametrize at `legacy/` (so the legacy KO-finisher test still covers the archived legacy files; the new format gets its own test in Task 8)

- [ ] **Step 1: Archive the 5 legacy scripts**

```
mkdir -p pixel_battle/data/scripts/legacy
git mv pixel_battle/data/scripts/01_lux_kite_garen.yaml
        pixel_battle/data/scripts/legacy/01_lux_kite_garen.yaml
git mv pixel_battle/data/scripts/02_garen_rush_lux.yaml
        pixel_battle/data/scripts/legacy/02_garen_rush_lux.yaml
git mv pixel_battle/data/scripts/03_glass_slow_brick.yaml
        pixel_battle/data/scripts/legacy/03_glass_slow_brick.yaml
git mv pixel_battle/data/scripts/04_yasuo_duel_ashe.yaml
        pixel_battle/data/scripts/legacy/04_yasuo_duel_ashe.yaml
git mv pixel_battle/data/scripts/05_lux_barrier_yasuo.yaml
        pixel_battle/data/scripts/legacy/05_lux_barrier_yasuo.yaml
```

- [ ] **Step 2: Update `test_script_library.py` to point at `legacy/`**

Open `pixel_battle/tests/test_script_library.py`. The file currently globs `pixel_battle/data/scripts/*.yaml`. Change it to glob `pixel_battle/data/scripts/legacy/*.yaml`:

```
# at the top of the file, locate the glob/dir constant and update:
SCRIPT_DIR = Path("pixel_battle/data/scripts/legacy")
```

(Read the file first to confirm the exact name of the dir constant or inline glob; update only that line and the parametrize source if needed.)

- [ ] **Step 3: Run the dump tool against each legacy file**

```
for n in 01_lux_kite_garen 02_garen_rush_lux 03_glass_slow_brick
         04_yasuo_duel_ashe 05_lux_barrier_yasuo; do
    python -m pixel_battle.scripts.dump_timeline \
        pixel_battle/data/scripts/legacy/${n}.yaml \
        pixel_battle/data/scripts/${n}.yaml
done
```

After this step, the 5 timeline candidates exist at `pixel_battle/data/scripts/*.yaml`.

- [ ] **Step 4: Polish each timeline file**

For each of the 5 timeline candidates: open the file and apply the following:

1. **Round `t` values to the nearest 100 ms.** A 3214 → 3200; a 3287 → 3300.
2. **Coalesce consecutive identical `do`** on the same character (the trace logs every transition; rounding may have produced duplicates).
3. **Trim out-of-range cast emissions** that appear at points where the character is clearly not in casting range (a cd cast at `dist >= 280` for a character whose cd skill has `range == 110` is the trace-tool capturing a no-op from the legacy run). Use judgment — the rounded timeline should read as the intended choreography, not the noisy raw trace.
4. **Add a prose chapter-header block at the top of the file** as YAML comments. Use this template — adapt to the matchup:

```
# ═══════════════════════════════════════════════════════
# 拉克絲 vs 蓋倫 — 風箏與追擊
# ═══════════════════════════════════════════════════════
# 【第一幕 · 開場  0.0–3.0s】
#   蓋倫低吼一聲,大劍出鞘,身形如鐵塊般壓進。
#   拉克絲足尖一點向後輕掠,光之法杖泛起淡金 —— 她需要距離。
#
# 【第二幕 · 蓄勢  3.0–7.5s】
#   首發《光之束縛》如鎖鏈般射出,蓋倫腳步一頓 ……
#
```

```
# 【第三幕 · 衝突  7.5–13.0s】
#   蓋倫一個閃現直插中庭,但 ……
#
# 【終幕 · 終結    13.0s+】
#   ……
# ──────────────────────────────────────────────────────────
```

The prose is for human readability — be vivid, match the actual choreography in the YAML below. Each act roughly corresponds to a span of timestamps.

5. **Set `duration_ms`** to the actual KO time from the trace (already done by the tool); leave the file as-is if the trace recorded the legacy KO time correctly.

- [ ] **Step 5: Verify each timeline file loads**

Run:

```
python -c "
from pathlib import Path
from pixel_battle.script.timeline_loader import load_timeline
for p in sorted(Path('pixel_battle/data/scripts').glob('*.yaml')):
    tl = load_timeline(p)
    print(f'{p.name}: {len(tl.left_events)} left,
        {len(tl.right_events)} right, dur={tl.duration_ms}')
"
```

Expected: 5 lines printed, no exceptions.

- [ ] **Step 6: Run full suite**

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 395 passed (the legacy KO-finisher test now points at `legacy/` which still has the `target_hp<=0` finishers; no regressions).

- [ ] **Step 7: Commit**

```
git add pixel_battle/data/scripts/
        pixel_battle/tests/test_script_library.py
git commit -m "feat(pixel-battle/scripts): convert 5 legacy scripts to
        timeline format"
```

Task 8: Integration + smoothness regression tests

Two parametrized integration tests over the 5 new timeline scripts:

1. **Library load + KO** — each timeline loads via the dispatcher, the env+driver headlessly runs to KO within `duration_ms × 1.2`.
2. **Smoothness regression** — no run of consecutive `idle` (action 0) emissions on either character's timeline exceeds 1000 ms. This is the direct test of the user's "卡卡" complaint; the legacy `INTENT_MAX_MS = 4000ms` stalls would fail it by 4×.

Files: – Create: `pixel_battle/tests/test_timeline_library.py`

☐ **Step 1: Write the tests**

```python
# pixel_battle/tests/test_timeline_library.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pathlib import Path

import pytest

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.script.loader import load_fight_file
from pixel_battle.script.timeline_driver import TimelineDriver


_SCRIPTS = sorted(Path("pixel_battle/data/scripts").glob("*.yaml"))
TICK_MS = 16


@pytest.mark.parametrize("path", _SCRIPTS, ids=lambda p: p.stem)
def test_loads_as_timeline_driver(path):
    driver = load_fight_file(path)
    assert isinstance(driver, TimelineDriver), (
        f"{path.name} did not dispatch to TimelineDriver")


@pytest.mark.parametrize("path", _SCRIPTS, ids=lambda p: p.stem)
def test_reaches_decisive_ko(path):
    driver = load_fight_file(path)
    tl = driver.timeline
    env = PixelBattleEnv(left_id=tl.left, right_id=tl.right)
    env.reset()
    budget_ms = int(tl.duration_ms * 1.2)
    while env.battle.elapsed_ms < budget_ms:
        left_act, right_act = driver.decide(env.battle)
        env.step(left_act, right_act)
        if env.battle.state.name == "KO":
            break
    assert env.battle.state.name == "KO", (
        f"{path.name} did not KO within {budget_ms} ms "
        f"(elapsed={env.battle.elapsed_ms}, "
```

```
        f"left_hp={env.battle.left.hp}, right_hp=
        {env.battle.right.hp})")
    loser = env.battle.left if env.battle.left.hp <= 0 else
        env.battle.right
    assert loser.hp <= 0


@pytest.mark.parametrize("path", _SCRIPTS, ids=lambda p: p.stem)
def test_no_idle_stretch_longer_than_1s(path):
    """Smoothness regression — directly tests the user's '卡卡'
        complaint.
    The legacy `INTENT_MAX_MS = 4000ms` stalls would fail this by
        4×."""
    driver = load_fight_file(path)
    tl = driver.timeline
    env = PixelBattleEnv(left_id=tl.left, right_id=tl.right)
    env.reset()
    left_idle_run = 0
    right_idle_run = 0
    left_max_run = 0
    right_max_run = 0
    while env.battle.elapsed_ms < int(tl.duration_ms * 1.2):
        left_act, right_act = driver.decide(env.battle)
        if left_act == 0:
            left_idle_run += TICK_MS
            left_max_run = max(left_max_run, left_idle_run)
        else:
            left_idle_run = 0
        if right_act == 0:
            right_idle_run += TICK_MS
            right_max_run = max(right_max_run, right_idle_run)
        else:
            right_idle_run = 0
        env.step(left_act, right_act)
        if env.battle.state.name == "KO":
            break
    assert left_max_run < 1000, (
        f"{path.name}: left timeline had a {left_max_run} ms idle
        stretch")
    assert right_max_run < 1000, (
        f"{path.name}: right timeline had a {right_max_run} ms idle
        stretch")
```

- [ ] **Step 2: Run the new tests**

Run: `python -m pytest pixel_battle/tests/test_timeline_library.py -v`
Expected: PASS — 5 × 3 = 15 tests passing.

If any test fails, the timeline polish in Task 7 was incomplete: – “did not KO” → the choreography doesn’t deal enough damage; add finisher events. – “idle stretch > 1000 ms” → the timeline has a gap; insert a filler movement

event. – “did not dispatch to TimelineDriver” → the YAML has neither timeline keys nor legacy script keys; check the converted file’s structure.

Iterate on the relevant YAML until all 15 pass.

- [ ] **Step 3: Run full suite**

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 395 + 15 = 410 passed.

- [ ] **Step 4: Validation render**

Render all 5 new timeline scripts:

```
for n in 01_lux_kite_garen 02_garen_rush_lux 03_glass_slow_brick
        04_yasuo_duel_ashe 05_lux_barrier_yasuo; do
    python -m pixel_battle.rl.play_scripted
        pixel_battle/data/scripts/${n}.yaml
done
```

Inspect each mp4 in `pixel_battle/output/scripted/`. Visually confirm: – No 4–second “stand still” stalls (the user’s “卡卡” complaint). – All 5 reach decisive KO. – The prose chapter headers in each YAML match the rendered choreography.

- [ ] **Step 5: Commit**

```
git add pixel_battle/tests/test_timeline_library.py
git commit -m "feat(pixel-battle/tests): timeline library — load, KO,
        smoothness regression"
```